



SASTRA
ENGINEERING · MANAGEMENT · LAW · SCIENCES · HUMANITIES · EDUCATION
DEEMED TO BE UNIVERSITY
(U/S 3 of the UGC Act, 1956)



THINK MERIT | THINK TRANSPARENCY | THINK SASTRA

OPERATING SYSTEMS LABORATORY

Course Code: CSE309R01

Semester: V

Lab Manual

2026

ROHITH KANNA GV

128003214

A-44

**SCHOOL OF COMPUTING
SHANMUGHA ARTS, SCIENCE, TECHNOLOGY AND RESEARCH
ACADEMY
(SASTRA Deemed to be University)
Tirumalaisamudram, Thanjavur-613 401**

Exercise No. 8. Simulation of Producer-Consumer problem

Write a program to simulate producer consumer problem

Objectives

Implementation of Producer-Consumer problem using bounded and unbounded

variations **Prerequisite / Tools Required (optional):**

Knowledge of Concurrency, Mutual exclusion, Synchronization and producer-consumer problem

Procedure / Algorithm

- ☐ Implement producer-consumer program with producers and consumers simulated as threads.
- ☐ Employ necessary semaphores for bounded and unbounded implementations
- ☐ Run the program to allow the producer and consumer share the buffer by synchronizing themselves through mutual exclusion

CODE:

Main.c

```
#include<stdio.h>
#include <stdlib.h>
int main() {
int c;
printf("\n\t\t.....Producer Consumer.....\n");
do {
printf("\n1. Producer\n2. Consumer\n3. Exit\nEnter choice: ");
scanf("%d", &c);
if (c == 1) {
system("gnome-terminal -- ./pd &");
} else if (c == 2) {
system("gnome-terminal -- ./cs &");
} else if (c != 3) {
printf("Invalid choice!\n");
}
} while (c != 3);
return 0;
}
```

Producer.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/shm.h>
#include <sys/ipc.h>
#define BUFFER_SIZE 5
int main() {
    key_t k1 = ftok(".", 65);
    key_t k2 = ftok(".", 66);
    key_t k3 = ftok(".", 67);
    int k1id = shmget(k1, sizeof(int), 0666 | IPC_CREAT);
    int k2id = shmget(k2, BUFFER_SIZE * sizeof(int), 0666 | IPC_CREAT);
    int k3id = shmget(k3, sizeof(int), 0666 | IPC_CREAT);
    int *mutex = (int*)shmat(k1id, NULL, 0);
    int *buffer = (int*)shmat(k2id, NULL, 0);
    int *index = (int*)shmat(k3id, NULL, 0);
    if (mutex == (void*)-1 || buffer == (void*)-1 || index == (void*)-1) {
        perror("shmat");
        exit(1);
    }
    *mutex = 0;
    *index = 0;
    int choice, value;
    printf("\n--- PRODUCER ---\n");
    do {
        if (*mutex == 0) {
            *mutex = 1;
            if (*index < BUFFER_SIZE) {
                printf("Enter value: ");
                scanf("%d", &value);
                buffer[*index] = value;
                (*index)++;
                printf("Produced. Buffer size: %d\n", *index);
            } else {
                printf("Buffer FULL!\n");
            }
            *mutex = 0;
        } else {
            printf("Consumer is working... Try later\n");
        }
        printf("Continue? (1/0): ");
        scanf("%d", &choice);
    } while (choice != 0);
    shmdt(mutex);
```

```

shmdt(buffer);
shmdt(index);
return 0;
}

```

Consumer.c

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/shm.h>
#include <sys/ipc.h>
#define BUFFER_SIZE 5
int main() {
    key_t k1 = ftok(".", 65);
    key_t k2 = ftok(".", 66);
    key_t k3 = ftok(".", 67);
    int k1id = shmget(k1, sizeof(int), 0666 | IPC_CREAT);
    int k2id = shmget(k2, BUFFER_SIZE * sizeof(int), 0666 | IPC_CREAT);
    int k3id = shmget(k3, sizeof(int), 0666 | IPC_CREAT);
    int *mutex = (int*)shmat(k1id, NULL, 0);
    int *buffer = (int*)shmat(k2id, NULL, 0);
    int *index = (int*)shmat(k3id, NULL, 0);
    if (mutex == (void*)-1 || buffer == (void*)-1 || index == (void*)-1) {
        perror("shmat");
        exit(1);
    }
    int choice;
    printf("\n--- CONSUMER ---\n");
    do {
        if (*mutex == 0) {
            *mutex = 1;
            if (*index > 0) {
                printf("Buffer before consume:\n");
                for (int i = 0; i < *index; i++) {
                    printf("%d ", buffer[i]);
                }
                printf("\nConsumed: %d\n", buffer[*index - 1]);
                (*index)--;
                printf("Buffer after consume:\n");
                for (int i = 0; i < *index; i++) {
                    printf("%d ", buffer[i]);
                }
                printf("\n");
            } else {
                printf("Buffer EMPTY!\n");
            }
        }
    } while (1);
}

```

```

*mutex = 0;
} else {
printf("Producer is working... Try later\n");
}
printf("Continue? (1/0): ");
scanf("%d", &choice);
} while (choice != 0);
shmdt(mutex);
shmdt(buffer);
shmdt(index);
return 0;
}

```

Output

The screenshot shows a Linux desktop with a terminal window open. The terminal displays the output of a producer-consumer program. The program prompts the user to choose between '1. Producer', '2. Consumer', and '3. Exit'. The user has chosen '1. Producer'. The program then prompts for a value to produce. The user enters '10', and the program outputs 'Produced. Buffer size: 1'. The program then prompts 'Continue? (1/0): 1'. The user enters '1', and the program outputs 'Enter value: 20'. The program then prompts 'Continue? (1/0): 1'. The user enters '1', and the program outputs 'Enter value: 30'. The program then prompts 'Continue? (1/0): 1'. The user enters '1', and the program outputs 'Enter value: 40'. The program then prompts 'Continue? (1/0): 1'. The user enters '1', and the program outputs 'Enter value: 50'. The program then prompts 'Continue? (1/0): 1'. The user enters '1', and the program outputs 'Buffer FULL!'. The program then prompts 'Continue? (1/0): 1'. The user enters '1', and the program outputs 'Continue? (1/0): 1'.

```

guest-wiq813@admin25-VirtualBox: ~
File Edit View Search Terminal Help
guest-wiq813@admin25-VirtualBox:~$ gcc pc.c -o pc
guest-wiq813@admin25-VirtualBox:~$ gcc producer.c -o producer
guest-wiq813@admin25-VirtualBox:~$ gcc consumer.c -o consumer
guest-wiq813@admin25-VirtualBox:~$ ./pc
.....Producer Consumer.....
1. Producer
2. Consumer
3. Exit
Enter choice: 1
1. Producer
2. Consumer
3. Exit
Enter choice: 1
Enter value: 10
Produced. Buffer size: 1
Continue? (1/0): 1
Enter value: 20
Produced. Buffer size: 2
Continue? (1/0): 1
Enter value: 30
Produced. Buffer size: 3
Continue? (1/0): 1
Enter value: 40
Produced. Buffer size: 4
Continue? (1/0): 1
Enter value: 50
Produced. Buffer size: 5
Continue? (1/0): 1
Buffer FULL!
Continue? (1/0): 1

```

```
guest-wlq813@admin25-VirtualBox: ~  
File Edit View Search Terminal Help  
guest-wlq813@admin25-VirtualBox:~$ gcc pc.c -o pc  
guest-wlq813@admin25-VirtualBox:~$ gcc producer.c -o pd  
guest-wlq813@admin25-VirtualBox:~$ gcc consumer.c -o cs  
guest-wlq813@admin25-VirtualBox:~$ ./pc  
.....Producer Consumer.....  
1. Producer  
2. Consumer  
3. Exit  
Enter choice: 1  
1. Producer  
2. Consumer  
3. Exit  
Enter choice: 2  
1. Producer  
2. Consumer  
3. Exit  
Enter choice:   
... CONSUMER ...  
Buffer before consume:  
10 20 30 40 50  
Consumed: 50  
Buffer after consume:  
10 20 30 40  
Continue? (1/0): 1  
Buffer before consume:  
10 20 30 40  
Consumed: 40  
Buffer after consume:  
10 20 30  
Continue? (1/0): 1  
Buffer before consume:  
10 20 30  
Consumed: 30  
Buffer after consume:  
10 20  
Continue? (1/0): 1  
Buffer before consume:  
10 20  
Consumed: 20  
Buffer after consume:  
10  
Continue? (1/0): 1  
Buffer before consume:  
10  
Consumed: 10  
Buffer after consume:  
10  
Continue? (1/0):   
C Tab Width: 8 Ln 10, Col 22 INS
```

```
guest-wlq813@admin25-VirtualBox: ~  
File Edit View Search Terminal Help  
guest-wlq813@admin25-VirtualBox:~$ gcc pc.c -o pc  
guest-wlq813@admin25-VirtualBox:~$ gcc producer.c -o pd  
guest-wlq813@admin25-VirtualBox:~$ gcc consumer.c -o cs  
guest-wlq813@admin25-VirtualBox:~$ ./pc  
.....Producer Consumer.....  
1. Producer  
2. Consumer  
3. Exit  
Enter choice: 1  
1. Producer  
2. Consumer  
3. Exit  
Enter choice: 2  
1. Producer  
2. Consumer  
3. Exit  
Enter choice: 3  
guest-wlq813@admin25-VirtualBox:~$   
sca }  
} while sl  
shndt(n sl  
shndt(b sl  
shndt(i ri  
return }  
C Tab Width: 8 Ln 10, Col 22 INS
```

Observation

Simulate producer consumer problem for ‘m’ producer and ‘n’ customer accessing a bounded buffer of limited size each in a separate window. Include the pseudo code for overflow and underflow conditions.

- **Multi-Window Simulation:** The simulation successfully runs by launching the main driver (`Main.c`), which spawns **m producers (`Producer.c`)** and **n consumers (`Consumer.c`)** as separate processes in distinct terminal windows using standard `gnome-terminal` commands.
- **Mutual Exclusion:** Shared memory variables (`mutex` and `index`) effectively coordinate the processes, ensuring that only one producer or consumer alters the bounded buffer at any given time.
- **Overflow Handling:** When the producer attempts to add elements beyond the fixed capacity of **5**, the boundary check prevents insertion and safely outputs a `Buffer FULL!` message.
- **Underflow Handling:** When the consumer attempts to retrieve data from an empty buffer, the condition check blocks consumption and safely outputs a `Buffer EMPTY!` message.

RESULT:

Thus successfully implemented producer - consumer problem